Validation Groups

- Implementing Validation Groups in Spring Boot
  - Step 1: Define Validation Groups
  - Step 2: Annotate Constraints with Groups
  - Step 3: Trigger Validation Using Groups
    - Method-Level Validation
    - Programmatic Validation
- Advanced Concepts
  - Group Sequences

Validation in software development ensures data integrity, making it a cornerstone of reliable applications. In Spring Boot, we often use annotations like `@NotNull`, `@Size`, or `@Email` for declarative validation. However, what if you need to validate different fields under different conditions?

For example:

- During user registration, validate only the username and password.
- During profile updates, validate additional fields like email and phone number.

Implementing Validation Groups in Spring Boot

Let's consider an example where we validate a `UserDTO` differently for registration and updates.

Step 1: Define Validation Groups

Create marker interfaces to represent validation contexts.

```
public interface OnRegistration {}
public interface OnUpdate {}
```

Step 2: Annotate Constraints with Groups

Use the `groups` attribute in validation annotations to associate them with specific groups.

```
import javax.validation.constraints.*;

public class UserDTO {

    @NotNull(groups = OnRegistration.class)
    @Size(min = 5, max = 15, groups = {OnRegistration.class, OnUpdate.class})
    private String username;

    @NotNull(groups = OnRegistration.class)
```

```java
    @Size(min = 8, groups = OnRegistration.class)
    private String password;

    @Email(groups = OnUpdate.class)
    private String email;

    @Pattern(regexp = "^\\d{10}$", groups = OnUpdate.class)
    private String phoneNumber;

    // Getters and Setters
}
```

Step 3: Trigger Validation Using Groups

Method-Level Validation

Use the `@Validated` annotation to specify the group at the controller or service layer.

```java
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/users")
public class UserController {

    @PostMapping("/register")
    public String registerUser(@Validated(OnRegistration.class) @RequestBody UserDTO userDTO) {
        // Registration logic
        return "User registered successfully!";
    }

    @PutMapping("/update")
    public String updateUser(@Validated(OnUpdate.class) @RequestBody UserDTO userDTO) {
        // Update logic
        return "User updated successfully!";
    }
}
```

Programmatic Validation

Use the `Validator` bean to perform group-specific validations programmatically.

```java
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import javax.validation.Validator;
import javax.validation.ConstraintViolation;
import java.util.Set;

@Service
public class UserService {

    @Autowired
    private Validator validator;

    public void validateForRegistration(UserDTO userDTO) {
        Set<ConstraintViolation<UserDTO>> violations = validator.validate(userDTO, OnRegistration.class);

        if (!violations.isEmpty()) {
```

```
            throw new IllegalArgumentException("Validation failed: " + violations);
        }
    }
}
```

Advanced Concepts

Group Sequences

Sometimes, you need to validate groups in a specific order. For example, validate basic constraints first and then domain-specific constraints.

You can use `@GroupSequence` to define the order.

```
import javax.validation.GroupSequence;

@GroupSequence({OnRegistration.class, OnUpdate.class})
public interface ValidationOrder {}
```

Now, use `ValidationOrder` as the group in `@Validated` or `validator.validate()`.